Aran Learn 21: How the Explainer Video Is Made

For: anyone who wants to change a line in the video, fix a picture, or make a new chapter. **Time:** about 1 hour to read and try one small change. **Before this:** Learn 13 (animation and visual explainers).

The video in `artifacts/Explainer video/` is not edited by hand in a video editor. It is built from four text files by three scripts, so anyone on the team can change a sentence and rebuild it the same way every time.

1. The big picture

```
script.txt  ──tts.py──►  one WAV per line  ──►  narration.wav + timing.json
                                                          │
player/*.js (scenes)  ◄──────────────────────────────────┘
      │
render.js: open the player in a headless browser, take one picture per frame
      │
ffmpeg: pictures → video, then add narration.wav  ──►  Aran - Explainer Video.mp4
```

The rule that holds everything together: **the voice decides the timing, and the pictures follow the voice.** Every animation is attached to a spoken line, not to a fixed second.

2. The script

`script.txt` is plain text. A line starting with `##` starts a chapter:

```
## checkdigit | 07 · Check digits | How a number checks itself
A check digit is one extra digit computed from all the others.
The subtitle text || The spoken text, when they need to differ
```

- Each line under the header is one spoken sentence and one subtitle.
- `text || say` lets the subtitle show "X25519" while the voice says it in a way a listener can follow.
- `tts.py` also has a small pronunciation table (for example Aran → "Uh-run", Verhoeff → "Ver-hoff") so names are read correctly.

3. The voice

`tts.py` uses **Kokoro**, a small open text-to-speech model (82 million parameters) that runs on the laptop's CPU through ONNX Runtime. Nothing is sent to an outside service.

- Voice `af_heart`, speed 0.95, 24 kHz audio.
- One WAV file per line, saved in `audio/`. If a WAV already exists it is reused, so changing one line only re-records that line.
- Silence at the start and end of each WAV is trimmed, then fixed gaps are added: 0.7 s before a chapter's first line, 0.45 s between lines, 1.3 s at the end of a chapter.
- The output is `narration.wav` (the whole voice track) and `timing.json`, which says exactly when each line starts and how long it lasts.

4. The player: a frame is a function of time

`player/player.html` is a 1920×1080 SVG drawing. The important idea is **seek**: calling `seek(t)` draws exactly what the video should show at second `t`, from scratch. It never depends on what was drawn before.

Why this matters:

- Rendering can jump to any frame, so four workers can each render a quarter of the video at the same time.
- A snapshot of second 600 always looks the same, so checking a frame is reliable.

Each chapter is a function in `player/scenes*.js` that builds its pictures once, then says when each one appears, in terms of the spoken lines:

```
X.show(card, 2, { type: 'up' });          // slides up when line 2 starts
X.show(arrow, 3, { f: 0.5, type: 'draw' }); // draws itself halfway through line 3
X.term(4, 'Check digit', 'one extra digit computed from the others');
```

- `k` is the line number inside the chapter, `f` is how far through that line (0 to 1).
- `X.term` puts a small glossary card in the top right whenever a technical word is spoken, so the word is shown and explained at the same moment.
- `X.each(fn)` runs custom code every frame, for things like the Feistel rounds or the Merkle tree, using `X.P` (smooth progress) and `X.L` (straight progress through a line).

Subtitles, the chapter title and the progress bar are drawn by `engine.js` from the same `timing.json`, so they can never drift from the voice.

5. Rendering

`render.js` opens the player in headless Chromium with Playwright, then for every frame:

1. calls `seek(frame / 30)`,
2. takes a JPEG screenshot,
3. pipes it straight into ffmpeg, which encodes H.264.

Four copies run at once over four frame ranges. The four parts are joined with ffmpeg's concat demuxer and the voice track is added as AAC. The subtitles are also written as a separate `.srt` file, so players can turn them on or off.

6. Try it: change one line

1. Edit the sentence in `script.txt`.
2. Delete that line's WAV in `audio/` (named `chapter_line.wav`, both numbers from 00).
3. Run `python3 tts.py`, then rebuild `timing.js` from `timing.json`.
4. Run `node snap.js check SECONDS` (or `node snap.js check chapter:line:0.85`) to save a picture of that moment and see any page errors.
5. Render with `node render.js START END 30 out.mp4` and join.

7. Checks before calling it done

- Snapshot every line at 85% of its length and look at the contact sheets: nothing overlaps, nothing is cut off at the edge.
- The browser console shows no errors for any chapter.
- The final MP4's length matches `timing.json`'s total within a frame, and the voice lines up with the subtitles at the start, the middle and the end.

Words used here

Word	Meaning
TTS	Text to speech: a program that turns written text into a voice.
ONNX	A file format for AI models that runs on many kinds of hardware without the original training code.
SVG	A drawing made of shapes described in text, so it stays sharp at any size.
Frame	One still picture. 30 frames per second makes smooth motion.
Headless browser	A real browser with no window, controlled by a script.
ffmpeg	A command-line tool that converts, joins and encodes audio and video.
H.264 / AAC	The most widely supported video and audio compression formats.
SRT	A plain-text subtitle file: number, start → end time, text.